

INF-08d Ve snaze o vyšší efektivitu navrhuji, řídím a hodnotím souběh procesů.

Na test máš 30 minut. Pracuj samostatně, bez pomoci druhých i umělé inteligence. Úlohy s kódem řeš bez spouštění; u úlohy 5 si odpověď můžeš po zapsání ověřit v Pythonu (Colab nebo Thonny). Povolená je kalkulačka. Odpovědi a postup zapiš do dokumentu nebo na papír a nahraj je do portfolia na Google Disku (soubor, fotka nebo sken).

1. Před online lekcí chceš stihnout: a) stáhnout z Disku 2 GB záznam minulé lekce; b) přečíst si Must know; c) vyplnit pracovní list podle Must know; d) nabít notebook; e) nahrát hotový list do portfolia. Které činnosti mohou probíhat souběžně a které musí jít po sobě? Zdůvodni.
2. Dvojice točí video pokusu do portfolia. Činnosti: A natočit pokus (20 min); B vybrat a stáhnout hudbu (10 min); C sestříhat video (15 min, až po A i B); D napsat popis a titulky (10 min, až po A); E export a nahrání do portfolia (5 min, až po C i D). a) Jak dlouho by práce trvala jednomu člověku? b) Navrhni rozvržení pro dva lidi (kdo, co a od kdy do kdy) s co nejkratší celkovou dobou. c) Jaké je zrychlení a proč nevyjde dvojnásobné?
3. Automatické testy mají zkontrolovat 900 odevzdaných programů. Test jednoho programu trvá 0,2 s a testy jsou na sobě nezávislé. Příprava (načtení seznamu, stažení zadání) a souhrnná zpráva trvají dohromady 20 s a rozdělit nejdou. Kolik trvá celá úloha na 1, 4 a 10 jádrech a jaké je zrychlení? Jakého zrychlení by šlo nejvýš dosáhnout při libovolném počtu jader?
4. Program napodobuje tři procesy, které současně přihlašují žáky na lekci. Kroky procesů se prostřídaly v pořadí ze seznamu poradi. Co program vypíše a kolik přihlášených by mělo být? Vysvětli, kde se jedna přihláška ztratila. Navrhni pořadí kroků, při kterém výsledek vyjde správně, a pravidlo, které správný výsledek zaručí vždy.

```
# Tři procesy (A, B, C) přihlašují žáky na online lekci.  
# Každý z nich udělá: načti počet přihlášených – přičti 1 – zapiš.  
pocet = 10  
nacteno = {}  
poradi = ["A čte", "B čte", "A zapisuje", "C čte", "B zapisuje", "C zapisuje"]  
for krok in poradi:  
    proces, akce = krok.split()  
    if akce == "čte":  
        nacteno[proces] = pocet  
    else:  
        pocet = nacteno[proces] + 1  
print(pocet)
```

5. Co program vypíše? Vysvětli, co dělají v programu start(), join() a with zamek. Co by se mohlo stát, kdybys a) vynechal zamek, b) vynechal cyklus s join()?

```
import threading  
  
pocet = 0  
zamek = threading.Lock()  
  
def pridej(n):  
    global pocet  
    for _ in range(n):  
        with zamek:  
            pocet += 1  
  
vlakna = [threading.Thread(target=pridej, args=(10000,)) for _ in range(4)]  
for v in vlakna:  
    v.start()  
for v in vlakna:  
    v.join()  
print(pocet)
```

6. Tři procesory mají zpracovat sedm nezávislých úloh s délkami 9, 7, 6, 5, 4, 3 a 2 min. a) Použij pravidlo: úlohy seřaď od nejdelší a každou dej procesoru, který se nejdříve uvolní. Kdy bude všechno hotové? b) Existuje lepší rozdělení? Pokud ano, uveď ho. c) Jaký je nejkratší možný čas a jak to zdůvodníš?

7. U kterých úloh souběžné zpracování na více jádrech (nebo počítačích) výrazně pomůže a u kterých ne? Zdůvodni. a) zmenšení 500 fotek; b) výpočet 1 000. členu posloupnosti, kde každý člen se počítá z předchozího; c) vyzkoušení všech 4místných kódů k zapomenutému heslu vlastního archivu; d) kopírování souboru z jednoho pomalého flash disku; e) vykreslení snímku 3D animace po částech obrazu.
8. Proces P1 zamkne soubor dochazka.csv a čeká, až se uvolní soubor lekce.csv. Proces P2 mezitím zamkl lekce.csv a čeká na dochazka.csv. Co nastane? Navrhni dvě pravidla, která tomu předejdou.

Test slouží k doložení výstupu do portfolia. Nehodnotí se body ani známkou – rozhoduje, jestli splníš kritéria na konci dokumentu.

1. **Souběžně: stahování a) a nabíjení d) běží samy a mohou probíhat současně s čímkoli. Po sobě musí jít b) → c) → e): list vyplníš až po přečtení Must know a nahrát ho můžeš, až je hotový (každý krok potřebuje výsledek předchozího).** – ověřuje: rozpoznání nezávislých částí práce, které mohou běžet souběžně, a závislostí, které souběh omezují
2. **a) 60 min; b) např. 1. člověk: A 0–20, C 20–35, E 35–40; 2. člověk: B 0–10, D 20–30 → hotovo za 40 min; c)**
 $S = \frac{60}{40} = 1,5$; **víc nejde, protože řetězec A → C → E musí jít po sobě a trvá 40 min (2. člověk mezitím čeká na A).** – ověřuje: návrh souběžného plánu s dodržení návazností a hodnocení zrychlení
 1. Jeden člověk: $20 + 10 + 15 + 10 + 5 = 60$ min.
 2. Nejdelší řetězec závislých činností: A → C → E, tedy $20 + 15 + 5 = 40$ min – rychleji to nejde ani s více lidmi.
 3. B nezávisí na ničem, dá se udělat během natáčení (0–10 min).
 4. C i D mohou začít ve 20. minutě po A; každý dělá jednu z nich (C 20–35, D 20–30).
 5. E začne až po C i D, tedy v 35. minutě, konec ve 40. minutě.
 6. Zrychlení $S = \frac{60}{40} = 1,5$.
3. **1 jádro: 200 s; 4 jádra: 65 s, $S \doteq 3,1$; 10 jader: 38 s, $S \doteq 5,3$; nejvýš $S = \frac{200}{20} = 10$ – nedělitelných 20 s zůstane vždy.** – ověřuje: hodnocení efektivity souběhu výpočtem zrychlení a vliv nedělitelné části práce
 1. Dělitelná část: $900 \cdot 0,2 = 180$ s, nedělitelná 20 s.
 2. $t_1 = 20 + 180 = 200$ s.
 3. $t_4 = 20 + \frac{180}{4} = 65$ s, $S = \frac{200}{65} \doteq 3,1$.
 4. $t_{10} = 20 + \frac{180}{10} = 38$ s, $S = \frac{200}{38} \doteq 5,3$.
 5. Pro velmi mnoho jader se t_p blíží 20 s, takže $S < \frac{200}{20} = 10$.
4. **Vypíše 12, správně má být 13. B zapisuje hodnotu, kterou načtl před zápisem A (10 + 1), a tím přepíše přihlášku A. Správně vyjde např. pořadí A čte, A zapisuje, B čte, B zapisuje, C čte, C zapisuje. Pravidlo: zámek (kritická sekce) – trojici načti–přičti–zapiš provede proces celou, než na počítadlo sáhne jiný.** – ověřuje: rozpoznání kolize nad sdílenými daty a návrh řízení souběhu
 1. A čte 10, B čte 10.
 2. A zapíše 11.
 3. C čte 11.
 4. B zapíše $10 + 1 = 11$ – přepsal zápis A.
 5. C zapíše $11 + 1 = 12$.
 6. Přihlásili se 3 žáci, správně je $10 + 3 = 13$.
5. **Vypíše 40000. start() spustí vlákno (4 vlákna běží souběžně), join() počká, až vlákno skončí, with zamek pustí k přičtení vždy jen jedno vlákno. a) Bez zámku se kroky „načti – přičti – zapiš“ mohou prostřídat a část přičtení se ztratí – výsledek může (ale nemusí) vyjít menší než 40000 a běh od běhu se liší. b) Bez join() se print může provést dřív, než vlákna doběhnou, a vypíše se menší číslo.** – ověřuje: řízení souběhu v programu – spuštění, čekání na dokončení a ochrana sdílené proměnné zámek
6. **a) P1: 9, 3; P2: 7, 4, 2; P3: 6, 5 → hotovo za 13 min. b) Ano: {9, 3}, {7, 5}, {6, 4, 2} – každý procesor 12 min. c) 12 min: celkem je práce 36 min, na 3 procesory tedy aspoň $\frac{36}{3} = 12$ min, a to rozdělení b) dosahuje.** – ověřuje: návrh a hodnocení rozdělení práce mezi souběžné procesy
 1. Na začátku: $9 \rightarrow P1, 7 \rightarrow P2, 6 \rightarrow P3$.
 2. $5 \rightarrow P3$ (volný v 6. min, konec 11), $4 \rightarrow P2$ (volný v 7., konec 11), $3 \rightarrow P1$ (volný v 9., konec 12).
 3. $2 \rightarrow P2$ nebo $P3$ (volné v 11.), konec 13 → pravidlo dá 13 min.
 4. Dolní mez: součet $9 + 7 + 6 + 5 + 4 + 3 + 2 = 36$, $\frac{36}{3} = 12$ min.
 5. Rozdělení se součty 12, 12, 12: $9 + 3, 7 + 5, 6 + 4 + 2$.
7. **Pomůže: a), c), e) – práce se dá rozdělit na nezávislé části. Nepomůže: b) – každý krok čeká na výsledek předchozího; d) – úzkým hrdlem je rychlost disku, ne procesor.** – ověřuje: posouzení, kdy souběh zvýší efektivitu a kdy ne

8. **Uvážnutí (deadlock):** každý proces drží jeden soubor a čeká na druhý, žádný nemůže pokračovat. Pravidla např.: zamykat soubory vždy ve stejném pořadí (nejdřív `lekce.csv`, pak `dochazka.csv`); zamknout oba najednou, nebo žádný; čekat jen omezenou dobu, pak zámek uvolnit a zkusit to znovu. – ověřuje: rozpoznání uvážnutí a návrh pravidel pro řízení souběhu

Kritéria splnění výstupu

- Navrhne souběžné rozvržení práce, které dodrží všechny návaznosti, a určí celkovou dobu (úlohy 2 a 6 – aspoň jedna zcela správně, druhá nejvýš s drobnou chybou ve výpočtu).
- Spočítá zrychlení pro různý počet jader a vysvětlí, proč ho omezuje nedělitelná část práce (úloha 3); u úlohy 7 správně rozhodne aspoň 4 z 5 případů.
- U kolize nad sdílenými daty určí výsledek, vysvětlí, kde se zápis ztratil, a navrhne řízení souběhu (zámek, čekání na dokončení) – úlohy 4 a 5.
- Pozná uvážnutí a navrhne aspoň jedno pravidlo, které mu spolehlivě předejde (úloha 8).

Pro učitele: varianty a přizpůsobení

Obměny: jiné délky úloh v úloze 6 (např. 8, 7, 6, 5, 4, 3, 3 na 3 procesory), jiný počet programů a doba přípravy v úloze 3, jiné pořadí kroků v úloze 4. Pro 1. ročník (náhled) lze úlohu 5 nahradit Scratchem: dvě postavy, druhá má začít mluvit až po první – žák popíše řešení bloky „vyšli zprávu a čekej“ a „po obdržení zprávy“. Praktická varianta: žák spustí program z úlohy 5 v Colabu se zámkem i bez něj, víckrát, a výsledky porovná. Úlohu 2 lze zadat jako časovou osu v Excalidraw. Kratší verze na 15 minut: úlohy 2, 3, 4 a 8.

Výstup se probírá v lekcích: 1.+2. H60 Efektivita algoritmů (týden 30)